Sintaxis Strudel

- Funciona encadenando funciones: `note("c a f e").s("piano")`.

```
note("a3 c#4 e4 a4")
.s("sawtooth")
.cutoff(500)
//.delay(0.5)
.room(0.5)
```

- Puedes inventarte tu propia funcion encadenada con `register`.

```
const effectChain = register('effectChain', (pat) => pat
  .s("sawtooth")
  .cutoff(500)
  //.delay(0.5)
  .room(0.5)
)
note("a3 c#4 e4 a4").effectChain()
```

Mini Notation

Lenguaje personalizado diseñado para escribir patrones ritmicos usando poco texto.

Sintaxis

- `~` : Silencio
- `('lateral)`: Permite escribir strings de varias lineas
- `('')`: Solo una linea
- `(')`: String normal sin mininotation
- `note("c e g b)`: Para escribir distintas notas se separa con espacios (el tiempo total es un segundo asiq cada nota dura un cuarto). **Un ciclo = un segundo de primeras**
- `*numero`: Acelera el tempo. Si pones ∗ 2 se reproduce dos veces en el ciclo. `note("[e5 b4 d5 c5]*2")`
- `/numero`: Hace que ocupe *numero* ciclos. `note("[e5 b4 d5 c5]/2")`
- `< >`: Define el largo de una secuencia basada en numero de eventos (cada nota es un ciclo creo) `note("<e5 b4 d5 c5>")` = `note("[e5 b4 d5 c5]/4")`
- `[ ]`: lo que esta dentro del bracket cuenta como 1 sola nota (de las que estan fuera del bracket). `note("e5 [b4 c5] d5 [c5 b4 d5 e5]")`
- `,`: Que las notas suenen a la vez (acorde). `note("g3,b3,e4")`. Tocar varios acordes en una secuencia: `note("<[g3,b3,e4] [a3,c3,e4] [b3,d3,f#4] [b3,e4,g4]>*2")`
- `@`: Para especificar el alargamiento de un solo acorde o nota. `note("<[g3,b3,e4]@2 [a3,c3,e4] [b3,d3,f#4]>*2")`. `_` hace lo mismo
- `!`: Repite una nota sin acelerarla `note("<[g3,b3,e4]!2 [a3,c3,e4] [b3,d3,f#4]>*2")`
- `?`: Randomiza una secuencia, 50% de oportunidades de que aparezca la nota o no `note("[g3,b3,e4]*8?")`. Puedes añadir la probabilidad de random → `note("[g3,b3,e4]*8?0.1")`

- `|` : Eventos separados por | hace que la nota que salga de ellas sea random. `note("[g3,b3,e4] | [a3,c3,e4] | [b3,d3,f#4]")`

Ritmos Euclidios

- Con paréntesis después un evento crea un ritmo basado en tres parámetros: **beats, segments y offset**. `s("bd(3,8,0)")`
- **Beats**: Controla cuantos beats hay en el ciclo
- **Segments**: Controla el total de segmentos en los que los beats serán distribuidos.
- **Offset**: Controla la posicion de inicio para distribuir los beats. Es opcional. Se necesita un segundo ritmo para oir la diferencia. `s("bd(3,8,0), hh cp")`

Javascript API

Pattern Functions

- Todos los patrones en Mini Notation pueden escribirse con funciones. `note("c3 eb3 g3)` = `note(seq("c3", "eb3", "g3"))`

Limites de Mini Notation

Para mezclar ritmos individuales usamos funciones a parte de mini notation.

```
stack(
note("c2 eb2(3,8)").s('sawtooth').cutoff(800),
s("bd(5,8), hh*8")
)
```

Combinando patrones

```
cat(
"g3,b3,e4",
"a3,c3,e4",
"b3,d3,f#4",
"b3,e4,g4"
).note()
```

Es lo mismo que

```
cat(
stack("g3","b3","e4"),
stack("a3","c3","e4"),
stack("b3","d3","fs4"),
stack("b3","e4","g4")
).note()
```

Que es lo mismo que

```
note("<[g3,b3,e4] [a3,c3,e4] [b3,d3,f#4] [b3,e4,g4]>")
```

Con patrones JS se pueden hacer muchas mas cosas

function	mini	Definición
`cat(x, y)`	`"<x y>"`	Los items se concatenan cada uno en un ciclo
`seq(x, y)`	`"x y"`	Los items se concatenan en un solo ciclo
`stack(x, y)`	`"x,y"`	Los items se tocan a la vez y duran lo mismo
`stepcat([3,x],[2,y])`	`"x@3 y@2"`	Concatena notas pero cada una tiene una duracion dentro del ciclo (primer num)
`polymeter([a, b, c], [x, y])`	`"{a b c, x y}"`	Alinea los patrones. Estos se repiten hasta que todos entran en un ciclo
`silence`	`"~"`	No hace nada
`run n(run(4)).scale("C4:pentatonic")`	`n("0 1 2 3").scale("C4:pentatonic")`	Un patron discreto de numeros de 0 a n-1
`binary "hh".s().struct(binary(5))`	`"hh".s().struct("1 0 1")`	Crea un patron binario desde un numero
`binaryN "hh".s().struct(binaryN(55532, 16))`	`"hh".s().struct("1 1 0 1 1 0 0 0 1 1 1 0 1 1 0 0")`	Igual que binary pero en n bits de longitud

https://strudel.cc/learn/factories/

Modificadores de tiempo

function	mini	
`"x".slow(2)`	`"x/2"`	Alarga el tiempo a n ciclos
`"x".fast(2)`	`"x*2"`	Lo hace mas rapido
`"x".euclid(3,8)`	`"x(3,8)"`	Cambia la estructura de un patrón para dormar un ritmo euclideo
`"x".euclidRot(3,8,1)`	`"x(3,8,1)"`	Como euclid pero tiene un parametro adicional para rotar la secuencia
`"x".early(.1)).s()`		Hace que un patrón empiece antes
`"x".late(.1)).s()`		Hace que un patron empiece despues

function	mini	
`"x".clip("<.5 1 2>")`		Multiplica la duración dado un numero. Corta un sample si se pasa de la duración